LANGRAPH IMPLEMENTATION

1. Purpose

The purpose of this project is to develop a basic LangGraph-based conversational agent using Google Gemini via LangChain.
It is intended as a reference implementation for building and executing a simple, state-driven AI workflow.

2. Description

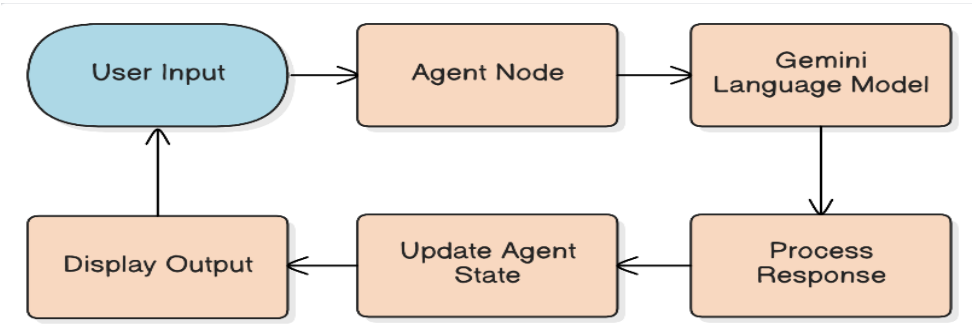
This project implements a single-node AI agent that:

- Accepts user input through the command line
- Sends the input to a Gemini language model
- Returns the generated response
- Calculates the response length

The agent workflow is managed using LangGraph, ensuring clear state handling and execution flow.

3. Workflow

1. User enters a prompt in the CLI
2. Input is stored in the agent state
3. LangGraph executes the agent node
4. Gemini generates a response
5. Response text and length are stored in state
6. Output is displayed to the user
7. The process repeats until the user types exit.



4. Architecture Overview

Agent Graph

START → Agent Node → END

- Entry Point: agent
- Total Nodes: 1

Agent State

Field	Description
user_input	User's query
agent_response	Model-generated reply
response_length	Character count of reply

5. Tech Stack

- **Language:** Python
- **Agent Framework:** LangGraph
- **LLM Framework:** LangChain
- **Model Provider:** Google Gemini
- **Model Used:** gemini-3-flash-preview
- **Interface:** Command Line (CLI)

6. Key Components

Agent State Definition

Defines the shared state used across the workflow.

LLM Initialization

Configures the Gemini model with deterministic output (temperature = 0).

Agent Node

- Sending user input to the LLM
- Extracting text from the response
- Computing response length

Graph Compilation

Compiles the LangGraph into an executable application.

7. Setup

Dependencies Installation

Install the required Python libraries using pip:

pip install langgraph langchain langchain-google-genai

API Key Configuration

This application requires a **Google Gemini API key**.

Set the API key as an environment variable before running the script.

- **macOS / Linux**

  export GOOGLE_API_KEY="your_api_key_here"

- **Windows (PowerShell)**

  setx GOOGLE_API_KEY "your_api_key_here"

The application reads the API key from the GOOGLE_API_KEY environment variable at runtime.

Note:
API keys should not be hardcoded in production environments for security reasons.

8. Execution

Input Example

Enter your question (type 'exit' to quit): Define LangGraph?

Output Example

LangGraph is an open-source library built on top of LangChain designed for building

stateful, multi-agent applications using Large Language Models (LLMs)…

Response Length: 3001 characters

Enter your question (type 'exit' to quit): exit

Exiting agent...

9. Limitations

- Single-node workflow
- No conversational memory
- No persistent storage
- No explicit error handling

10. Conclusion

This project provides a minimal and clear example of using LangGraph with Google Gemini to build a conversational agent.
It is best suited for learning, experimentation, and early prototyping.